Web 2 – Séance 02

Requêtes HTTP, Conventions REST et Documentation d’API

Rappel : Requêtes HTTP

- Verbes HTTP : GET, POST, PUT, PATCH, DELETE
- Chemin : `/recipes`, `/login`, `/users/:id`
- Paramètres de requête : `/recipes?servings=4`, `/users/:id?include=posts`
- Corps de la requête : données envoyées par le client au serveur
- Entêtes HTTP : `Content-Type`, `Authorization` etc.
- Codes de retour HTTP : 200, 201, 204, 400, 401, 403, 404, 500
- Corps de la réponse : données envoyées par le serveur au client

Rappel : Conventions REST

Conventions REST pour les verbes HTTP et les chemins d'accès aux ressources.
Permet de standardiser les API et de faciliter leur compréhension.

```
// GET : récupérer une ressource
GET /recipes           // Toutes les recettes
GET /recipes/5         // Recette avec id=5

// POST : créer une ressource
POST /recipes          // Créer une nouvelle recette
// Body: { title: "Pancakes", servings: 4 }

// PUT : remplacer complètement une ressource
PUT /recipes/5         // Remplacer la recette 5
// Body: { title: "Pancakes", servings: 6 }

// PATCH : modifier partiellement une ressource
PATCH /recipes/5       // Modifier partiellement la recette 5
// Body: { servings: 8 }

// DELETE : supprimer une ressource
DELETE /recipes/5      // Supprimer la recette 5
```



Rappel : Paramètres de requête

- Path parameters : `/recipes/:id` (ex: `/recipes/5`)
  - Utilisés pour identifier une ressource spécifique
- Query parameters : `?key1=value1&key2=value2` (ex: `/recipes?servings=4`)
  - Utilisés pour donner des précisions sur la requête
  - souvent filtres, tri, pagination, options facultatives

Rappel : Corps de la requête

Request body : données envoyées par le client au serveur.
Convention REST : données envoyées au format JSON.

- `GET` : pas de body
- `POST` : body correspondant à la ressource à créer
- `PUT` : body correspondant à la ressource complète à remplacer
- `PATCH` : body correspondant aux propriétés de la ressource à modifier
- `DELETE` : pas de body

⚠️ Ne pas oublier le header `Content-Type: application/json`

Rappel : Codes de retour HTTP

- `200 OK` : succès avec contenu
- `201 Created` : créé
- `204 No Content` : succès sans contenu
- `400 Bad Request` : erreur client, souvent données fournies invalides
- `401 Unauthorized` : non authentifié
- `403 Forbidden` : authentifié mais pas autorisé
- `404 Not Found` : ressource inexistante
- `409 Conflict` : conflit avec l'état actuel de la ressource, souvent doublon
- `500 Internal Server Error` : erreur serveur

Rappel : Corps de la réponse

Données envoyées par le serveur au client.
Convention REST : données envoyées au format JSON.

- `GET` : body correspondant à la ressource demandée
- `POST` : pas toujours un body
  - Si la ressource créée correspond exactement à la ressource demandée, pas de body nécessaire
  - Si un ID est généré, le body peut contenir la ressource créée avec son ID
- `PUT` : pas de body, sauf si certaines propriétés sont modifiées par le serveur (ex: date de modification)
- `PATCH` : pas de body
- `DELETE` : pas de body

Convention pour ce cours. Parfois un message de confirmation, parfois toujours la ressource modifiée/supprimée, parfois données précédentes, …

Rappel : Tests d'API

Extension VSCode : REST Client, permet de lancer des requêtes HTTP

```
### Récupérer toutes les recettes
GET http://localhost:3000/recipes


### Créer une nouvelle recette
POST http://localhost:3000/recipes
Content-Type: application/json

{
    "title": "Pancakes",
    "servings": 4,
    "ingredients": ["flour", "milk", "eggs"]
}
```



Rappel : Tests d'API (suite)

Il est possible de réaliser des tests d'API avancés avec REST Client, en utilisant des variables et des scripts.

```
### Créer une nouvelle recette
# @name createRecipe
POST http://localhost:3000/recipes
Content-Type: application/json

{
    "title": "Pancakes",
    "servings": 4,
    "ingredients": ["flour", "milk", "eggs"]
}

### Récupérer la recette créée
GET http://localhost:3000/recipes/{{createRecipe.response.body.id}}
```



Attention : cette syntaxe est spécifique à l'extension REST Client sur Visual Studio Code.

Documentation API

Permet de décrire les endpoints d'une API, leurs paramètres, leurs corps de requête et de réponse, ainsi que les codes de retour. Avec une documentation claire, les développeurs peuvent comprendre et utiliser l'API plus facilement sans aller lire le code source.

Formats de documentation :

- Commentaires JSDoc dans le code source
- **Document Markdown (README.md)**
- Fichier OpenAPI (Swagger)

Documentation API : JSDoc

Au sein du code source, au dessus de chaque route.

```
/**
 * @route GET /recipes
 * @summary Récupère toutes les recettes
 * @param {string} (query) order - Ordre de tri des recettes (asc|desc)
 * @returns {Array<Recipe>} 200 - Liste de toutes les recettes
 */
app.get('/recipes', (req, res) => {
    const recipes = getAllRecipes();
    res.json(recipes);
});
```



```
/**
 * @route POST /recipes
 * @summary Crée une nouvelle recette
 * @param {NewRecipe} (body) recipe - Nouvelle recette à créer
 * @returns {Recipe} 201 - Recette créée avec son ID
 * @returns 409 - Une recette avec le même titre existe déjà
 */
```



Documentation API : JSDoc (suite)

```
/**
 * @route PUT /recipes/:id
 * @summary Met à jour une recette existante
 * @param {number} (path) id - ID de la recette à mettre à jour
 * @param {Recipe} (body) recipe - Recette à mettre à jour
 * @returns 204 - Recette mise à jour
 * @returns 404 - Recette non trouvée
 */
```



```
/**
 * @route DELETE /recipes/:id
 * @summary Supprime une recette existante
 * @param {number} (path) id - ID de la recette à supprimer
 * @param {string} (header) Authorization - Token d'authentification JWT
 * @returns 204 - Recette supprimée
 * @returns 401 - Non authentifié
 * @returns 403 - Non autorisé
 * @returns 404 - Recette non trouvée
 */
```



Documentation API : Document Markdown

Fichier `README.md` ou `API.md` pour documenter l'API.

```
# API MiamMiam

## GET /recipes
- Description : Récupère toutes les recettes
- Paramètres :
  - order=asc|desc (query) : Ordre de tri des recettes (optionnel)
- Réponses :
  - 200 : Succès
    - Body : Liste de toutes les recettes

## POST /recipes
- Description : Crée une nouvelle recette
- Body : Nouvelle recette à créer
- Réponses :
  - 201 : Recette créée
    - Body : Recette créée avec son ID
  - 409 : Une recette avec le même titre existe déjà
```

Documentation API : Document Markdown (suite)

```
## PUT /recipes/:id
- Description : Met à jour une recette existante
- Body : Recette à mettre à jour
- Paramètres :
  - id (path) : ID de la recette à mettre à jour
- Réponses :
  - 204 : Recette mise à jour
  - 404 : Recette non trouvée

## DELETE /recipes/:id
- Description : Supprime une recette existante
- Authentification : JWT
  - Utilisateurs administrateurs
- Paramètres :
  - id (path) : ID de la recette à supprimer
- Réponses :
  - 204 : Recette supprimée
  - 401 : Non authentifié
  - 403 : Non autorisé
  - 404 : Recette non trouvée
```



Documentation API : OpenAPI (Swagger)

Fichier `openapi.yaml` pour documenter l'API.
Éditable/visualisable avec Swagger Editor : https://editor.swagger.io/

```yaml
openapi: 3.0.0
info:
  title: API MiamMiam
  version: 1.0.0
paths:
  /recipes:
    get:
      summary: Récupère toutes les recettes
      parameters:
        - name: order
          in: query
          description: Ordre de tri des recettes (asc|desc)
          required: false
          schema:
            type: string
            enum: [asc, desc]
      responses:
        '200':
          description: Liste de toutes les recettes
```

Récapitulatif Séance 02

- Requêtes HTTP
- Conventions REST
- Documentation API
  - JSDoc
  - Document Markdown
  - OpenAPI (Swagger)

Prochaine séance : Séance 03 — JWT et authentification.

Exercice filé S02 partie 1

- Dans le backend du projet MiamMiam, créez un fichier `API.md` pour documenter l'API REST.
- Documentez l'entièreté des routes existantes
- **Optionnel** : Créez un fichier `openapi.yaml` pour documenter l'API REST avec OpenAPI (Swagger).

Exercice filé S02 partie 2

- Implémentez la route PATCH `/recipes/:id`.
  - Cette route permet de mettre à jour partiellement une recette existante.
  - La spécification de la route doit être :

```
/**
 * @route PATCH /recipes/:id
 * @summary Met à jour partiellement une recette (auteur ou admin uniquement)
 * @param {number} id.path - L'ID de la recette
 * @param {UpdatedRecipeDTO} req.body - Les données de la recette à mettre à jour
 * @returns {RecipeDTO} 200 - La recette mise à jour
 * @returns {400} - ID invalide ou données invalides
 * @returns {401} - Non autorisé
 * @returns {403} - Accès refusé
 * @returns {404} - Recette non trouvée
 */
```



N’oubliez pas de mettre à jour la documentation et de tester vos routes.